

Módulo 10 – Capítulo 10 – Sección 01

Deuda técnica en plataformas de IA: identificación, cuantificación y gestión planificada

La deuda técnica es un concepto familiar en el desarrollo de software: la acumulación de decisiones de diseño que funcionan en el corto plazo pero que generan costos crecientes de mantenimiento y extensión en el largo plazo. En plataformas de IA, la deuda técnica tiene una característica que la hace especialmente peligrosa: su impacto se multiplica. Un componente con alta deuda técnica en una aplicación web afecta al equipo que lo mantiene; el mismo nivel de deuda en el feature store, el model registry, o el LLM Gateway de una plataforma de IA puede bloquear simultáneamente a todos los equipos que dependen de ese componente cuando falla, convirtiendo un problema técnico local en un incidente de productividad organizacional. Esta asimetría entre la ubicación de la deuda y la escala de su impacto es la razón por la que la gestión planificada de la deuda técnica en plataformas de IA no es opcional: es una obligación operacional del equipo de plataforma.

Los **tipos de deuda técnica más comunes** en plataformas de IA son distintos de los que aparecen en aplicaciones de negocio convencionales, porque el stack tecnológico es más volátil y las dependencias entre componentes son más estrechas. El **dependency drift** es la categoría más prevalente: el ecosistema de Python para ML (transformers, torch, numpy, scikit-learn, cuda) evoluciona a una velocidad que hace que las dependencias sin pinning explícito se rompan silenciosamente con cada actualización del entorno base. Un pipeline de entrenamiento sin `requirements.txt` con versiones exactas pinned (via `pip-compile` o `poetry.lock`) es un pipeline que fallará de forma no reproducible en el momento menos conveniente. La **observabilidad incompleta** es la deuda que más cuesta durante los incidentes: componentes de plataforma sin métricas en Prometheus, sin logs estructurados con `request_id` para correlación, o sin health checks semánticos (que verifican que el componente hace lo que debe, no solo que está vivo). El MTTR de incidentes en componentes con observabilidad incompleta supera en 3-5x el MTTR de componentes bien instrumentados, porque el diagnóstico requiere investigación manual en lugar de seguir las correlaciones del dashboard.

La **cuantificación de la deuda técnica** requiere métricas proxy que hagan visible lo que de otra manera permanece invisible en el backlog. El **MTTR por componente** (Mean Time to Recovery) es el indicador más fiable de deuda en observabilidad: si reparar un incidente del model registry tarda sistemáticamente más de 30 minutos, hay instrumentación insuficiente. La **cognitive complexity de los módulos críticos** —medida con herramientas como `radon` para Python o SonarQube— señala el código que más tiempo consume en on-boarding de nuevos ingenieros y que más probabilidad tiene de introducir regresiones en cambios. La **ratio de PRs de mantenimiento vs PRs de nuevas capacidades** en el repositorio de la plataforma es un indicador de la salud del balance entre deuda y evolución: cuando más del 50% del capacity del equipo se gasta en mantenimiento en lugar de nuevas capacidades, la plataforma está en estado de deterioro activo. La **matriz de impacto × esfuerzo** aplicada al backlog de deuda técnica permite priorizar: alta deuda + alto impacto (afecta a muchos equipos o causa incidentes frecuentes) + bajo esfuerzo de resolución = primera prioridad en el sprint de deuda.

La **gestión planificada** de la deuda sigue el modelo que Stripe llama "innovation tokens": reservar un porcentaje fijo del capacity del equipo —típicamente entre el 20% y el 30%— para reducción de deuda técnica, documentado en el roadmap con la misma visibilidad que las nuevas capacidades y con la misma protección contra la presión de entregar features adicionales. Sin esta reserva explícita, la deuda siempre pierde el debate contra el feature de negocio urgente: la deuda tiene costos futuros difusos mientras que el feature tiene un stakeholder con nombre y apellido que pregunta por él cada semana. El roadmap de deuda publicado en el developer portal hace visible el compromiso del equipo de plataforma con la sostenibilidad técnica, y construye la credibilidad necesaria para que los equipos consumidores confíen en que los problemas sistémicos que reportan serán resueltos en un plazo razonable.

Tipos de deuda técnica específicos de plataformas de IA

- **Dependency drift:** dependencias de Python/CUDA sin versiones pinned que se rompen silenciosamente con actualizaciones del entorno base; requiere dependency lockfiles (`pip-compile`, `poetry.lock`) y un pipeline de CI que testea la compatibilidad en cada merge a main.
- **Observabilidad incompleta:** componentes sin métricas en Prometheus, sin logs estructurados con `request_id`, o sin health checks semánticos; detectado cuando el MTTR de incidentes supera 30 minutos o cuando el diagnóstico requiere SSH al pod para inspeccionar el estado interno.

- **Configuración hardcodeada:** valores de configuración (bucket names, endpoint URLs, thresholds de alerta) embebidos en el código en lugar de inyectados via environment variables o ConfigMaps; cada cambio de configuración requiere redeploy y aumenta el riesgo de errores de copy-paste entre entornos.
- **Abstracciones con fugas:** los consumidores de la plataforma necesitan entender detalles de Kubernetes, Helm o AWS IAM para usar las abstracciones del SDK interno; síntoma de que el golden path no encapsula suficiente complejidad y que el modelo de "progressive disclosure" no está completamente implementado.
- **Tests de integración ausentes:** los componentes de la plataforma son testeados unitariamente pero no como sistema integrado (registry + serving + monitoring + gateway); los bugs de integración solo se detectan en producción, con impacto en todos los equipos que usan esa combinación de componentes.

Nota del Arquitecto: La deuda técnica en una plataforma de IA es deuda con interés compuesto: cada semana que pasa sin resolverla, más equipos construyen encima de los comportamientos incorrectos asumiendo que son el contrato estable. Cuando eventualmente se resuelve la deuda, hay que coordinar la migración de todos los equipos que adaptaron su código al comportamiento incorrecto, multiplicando el costo de resolución. Resolver temprano es siempre más barato que resolver tarde.

La identificación y cuantificación de la deuda técnica convierte un problema invisible en un backlog priorizado y gestionable. La siguiente sección examina el proceso de migración de modelos —una de las operaciones de mayor riesgo en plataformas de IA— donde la deuda técnica en el pipeline de deployment se manifiesta con más claridad y consecuencias más directas.

Módulo 10 – Capítulo 10 – Sección 02

Migraciones de modelos: actualizar modelos en producción sin interrumpir a los equipos usuarios

Una migración de modelo es el proceso de reemplazar un modelo en producción por una versión más nueva —sea un ajuste de fine-tuning, una actualización de arquitectura, un cambio de proveedor, o una transición entre versiones MAJOR con cambios breaking en el schema de respuesta— de forma que los equipos consumidores del endpoint no experimenten interrupciones de servicio, degradaciones de calidad inesperadas, ni se vean obligados a hacer cambios de emergencia en su código. Esta definición parece razonable en el papel; su complejidad se revela en la práctica: los modelos de IA tienen propiedades que los hacen más difíciles de migrar que una API REST convencional. El comportamiento de un modelo no es completamente especificable por un schema —dos modelos pueden retornar respuestas válidas según el schema pero con tonos, longitudes, o razonamientos diferentes que rompen las asunciones implícitas de las aplicaciones que los consumen. La única forma de saber con certeza si el nuevo modelo se comporta de forma compatible con lo que los equipos consumidores esperan es exponerlo a tráfico real de producción y comparar.

La estrategia de migración apropiada depende de la magnitud del cambio, siguiendo el esquema de versionado semántico de modelos que el Capítulo 03 estableció. Para actualizaciones **PATCH** (mismo modelo, mismos pesos, ajuste mínimo de parámetros de serving) y **MINOR** (fine-tuning adicional o mejora de calidad sin cambios breaking en el output schema), el mecanismo es un **rolling deploy transparente**: el equipo de plataforma actualiza el modelo en el endpoint existente con zero-downtime usando rolling update de Kubernetes, los equipos consumidores no necesitan hacer nada, y el rollback es inmediato restaurando la imagen del contenedor anterior. Para actualizaciones **MAJOR** (cambio de arquitectura, cambio de proveedor, o cambios breaking en el formato de respuesta que requieren ajustes en el código del consumidor), el proceso es significativamente más deliberado: el nuevo modelo se despliega en un endpoint paralelo con versión explícita en la URL (`/v2/completions` junto al `/v1/completions` existente), los equipos consumidores son notificados con al menos 30 días de antelación, se ofrece un período de coexistencia de ambas

versiones, y la versión anterior se deprecia solo cuando todos los consumidores han completado la migración o cuando el período de deprecación ha vencido.

El **shadow mode** es el mecanismo de validación de mayor valor para migraciones de modelos con riesgo alto —cambio de proveedor, upgrade de versión MAJOR, o cualquier cambio donde no hay suficiente confianza en que el comportamiento sea equivalente. En shadow mode, el nuevo modelo procesa el 100% del tráfico de producción en paralelo con el modelo actual: ambos reciben exactamente el mismo request, pero solo la respuesta del modelo actual se retorna al usuario. Las respuestas del modelo en shadow se almacenan en un topic de Kafka o un bucket de S3 para análisis diferido, y un job de evaluación compara automáticamente las respuestas de ambos modelos usando las métricas de calidad definidas (ROUGE, coherencia, fidelidad, o evaluación por LLM-as-a-judge). Si las métricas del modelo en shadow son iguales o mejores que las del modelo actual durante 24-72 horas de tráfico representativo, hay confianza suficiente para proceder con el canary rollout.

El **canary rollout** progresivo reduce el riesgo de la migración al limitar la exposición inicial del nuevo modelo a una fracción pequeña del tráfico real. El protocolo estándar de la industria para migraciones de modelos de riesgo medio-alto es: 5% del tráfico al nuevo modelo durante 24 horas de monitoreo intensivo → 10% durante otras 24 horas → 25% → 50% → 100%. En cada etapa se comparan las métricas de calidad y performance del nuevo modelo vs el modelo actual: si la latencia P95 del nuevo modelo supera la del actual en más del 20%, o si el error rate es más de 1% superior, el sistema activa un **rollback automático** restaurando el tráfico completo al modelo anterior y notificando al equipo de plataforma para investigación. El umbral de rollback automático es deliberadamente conservador en las primeras etapas del canary: es mejor detectar un falso positivo (rollback innecesario) que pasar un problema real al 50% del tráfico.

Etapas de una migración de modelo segura

- **Preparación en Staging:** publicar el nuevo modelo en Staging con documentación completa de cambios respecto a la versión anterior, guía de migración detallada con ejemplos de código si hay breaking changes, y período de disponibilidad en Staging (mínimo 5 días laborables) para que los equipos consumidores hagan testing de integración antes del rollout a producción.
- **Shadow mode (obligatorio para MAJOR, opcional para MINOR):** el nuevo modelo procesa el 100% del tráfico en paralelo sin retornar respuestas al usuario; las

respuestas se almacenan para comparación diferida; el período de shadow es de 24-72 horas con tráfico representativo de diferentes horarios del día.

- **Canary rollout progresivo:** 5% → 10% → 25% → 50% → 100% del tráfico con observación de 24-48 horas en cada paso; rollback automático si la latencia P95 supera el 20% o el error rate supera 1% por encima del baseline del modelo actual.
- **Monitoring intensivo post-migración:** durante las primeras 48 horas post-migración completa, los umbrales de alerta se reducen al 50% de los valores normales para detectar degradaciones sutiles antes de que escalen; el equipo de plataforma mantiene disponibilidad reforzada durante este período.
- **Retención del endpoint anterior:** después de la migración completa, el endpoint del modelo anterior permanece disponible durante 30 días para que los equipos rezagados completen la migración; pasado ese período, el endpoint se elimina y el modelo se archiva en el registry con estado `Archived`.

Nota del Arquitecto: El shadow mode requiere que la infraestructura de inferencia pueda procesar el doble del tráfico durante el período de validación, porque ambos modelos reciben el 100% de los requests en paralelo. Este costo de infraestructura temporal —que puede ser significativo para modelos grandes— es la prima de seguro que permite validar el comportamiento del nuevo modelo con tráfico real de producción antes de exponer a ningún usuario. Para migraciones críticas, este costo se justifica ampliamente: el costo de un incidente de calidad en producción con el 100% del tráfico es siempre superior al costo de ejecutar shadow mode durante 72 horas.

La migración de modelos segura requiere infraestructura técnica —shadow mode, canary rollout, rollback automático— y proceso organizacional —comunicación adelantada, período de staging, monitoreo intensivo post-migración. La siguiente sección examina el ciclo de vida completo de los componentes de la plataforma, desde su introducción hasta su deprecación, con el mismo nivel de cuidado que las migraciones de modelos individuales requieren.

Módulo 10 – Capítulo 10 – Sección 03

Deprecación de componentes: el ciclo de vida de las APIs y abstracciones de plataforma

Una plataforma de IA en operación no es un conjunto de componentes estáticos: es un sistema vivo que evoluciona, donde las abstracciones que fueron la mejor decisión de diseño hace dos años pueden ser hoy las que bloquean la adopción de mejores patrones. El feature store v1 que fue construido con una arquitectura de batch daily update puede necesitar ser reemplazado por uno con capacidades de streaming en tiempo real; la CLI de la plataforma con sintaxis verbose puede necesitar ser reemplazada por una con sintaxis más ergonómica; el endpoint de serving que usa gRPC puede necesitar ser complementado o reemplazado por uno que use REST+OpenAI-compatible para facilitar la migración desde proveedores externos. En todos estos casos, la organización enfrenta la misma tensión: la plataforma necesita evolucionar para seguir siendo relevante, pero los equipos que construyeron encima de los componentes anteriores necesitan tiempo para adaptarse sin que el piso desaparezca debajo de ellos.

La deprecación de componentes de plataforma sigue el mismo principio de contrato social que la deprecación de APIs en software de negocio: una vez que un componente es expuesto a los equipos consumidores y éstos construyen sobre él, eliminarlo unilateralmente es un breaking change que afecta a múltiples equipos simultáneamente. La diferencia entre deprecación bien y deprecación mal no está en si el componente se elimina —todos los componentes eventualmente se eliminan— sino en la calidad del proceso que acompaña la eliminación: cuánto tiempo de antelación se da, qué soporte de migración se ofrece, y cómo se comunica el proceso. Una deprecación bien ejecutada puede convertirse en una oportunidad para estrechar la relación entre el equipo de plataforma y los equipos consumidores; una deprecación abrupta puede generar desconfianza que dura años y hace que los equipos eviten adoptar nuevas abstracciones por miedo a quedar bloqueados de nuevo.

El **proceso formal de deprecación** estructura el ciclo de vida del componente en cuatro fases con duraciones mínimas definidas. La **fase de anuncio** se inicia con al menos 90 días de antelación a la fecha de sunset: se publica en el developer portal la intención de deprecación,

el motivo, la fecha de sunset, y el componente de reemplazo con documentación completa; se envía notificación directa a los responsables técnicos de cada equipo consumidor identificado en los usage analytics; y se añade el header HTTP estándar `Deprecation: true` y `Sunset: 2025-06-01` a todas las respuestas del componente deprecado. La presencia de estos headers en los responses permite que los clientes HTTP instrumentados (y los ingenieros que monitorean sus logs) detecten automáticamente que están usando un componente en proceso de deprecación. La **fase de migración asistida** ocupa los primeros 60 días del período de 90: el equipo de plataforma ofrece pairing sessions individuales con los equipos consumidores, guías de migración con ejemplos de código específicos para los patrones de uso más comunes, y un canal de Slack dedicado con SLA de respuesta de 4 horas para preguntas de migración. La **fase read-only** comienza a los 60 días del anuncio: el componente sigue funcionando para operaciones de lectura (`getModel`, `listExperiments`, `queryFeature`) pero rechaza nuevas escrituras (`registerModel`, `logMetric`, `writeFeature`), con respuesta HTTP 405 Method Not Allowed que incluye en el body la URL del componente de reemplazo. Esta fase da tiempo a los equipos que aún no han completado la migración para extraer sus datos históricos sin urgencia de tiempo real. La **fase de sunset** aplica cuando se alcanza la fecha de sunset: el endpoint retorna HTTP 410 Gone con un body JSON estructurado que incluye el mensaje de error, la URL del componente de reemplazo, y el enlace a la guía de migración — nunca HTTP 404, que comunica "no existe" en lugar de "existía y se retiró intencionalmente".

El **post-mortem de adopción** después de cada deprecación es la práctica que convierte la experiencia en aprendizaje institucional: cuántos equipos completaron la migración antes de la fecha de sunset (target: >95%), cuántos requirieron extensión del período y por qué, qué barreras de migración se encontraron más frecuentemente, y qué cambios en el proceso o en la calidad de la documentación habrían reducido el friction de migración. Esta información retroalimenta el diseño de los próximos componentes para minimizar la necesidad de deprecaciones breaking en el futuro —el mejor proceso de deprecación es el que no necesita ejecutarse porque el diseño original fue suficientemente extensible.

Fases del proceso de deprecación de componentes

- **Anuncio formal (90 días antes):** publicación en developer portal con motivo, fecha de sunset y componente de reemplazo; notificación directa a responsables técnicos de equipos consumidores identificados por usage analytics; headers HTTP `Deprecation: true` y `Sunset: <fecha>` en todas las respuestas del componente.

- **Migración asistida (días 1-60):** pairing sessions con equipos consumidores bajo solicitud; guías de migración con ejemplos de código para los patrones de uso más comunes; canal de Slack dedicado con SLA de respuesta de 4 horas para preguntas de migración durante el período de transición.
- **Read-only phase (días 60-90):** el componente acepta operaciones de lectura pero rechaza escrituras con HTTP 405 y body con la URL del componente de reemplazo; permite extraer datos históricos sin presión de tiempo real para equipos que aún no completaron la migración completa.
- **Sunset enforcement (día 90+):** el endpoint retorna HTTP 410 Gone con body JSON estructurado que incluye el mensaje de error, la URL del componente de reemplazo, y el enlace a la guía de migración; nunca HTTP 404 ni eliminación silenciosa sin respuesta informativa.
- **Post-mortem de adopción:** análisis post-deprecación de qué porcentaje de equipos migró antes del sunset, cuántos requirieron extensión, qué barreras se encontraron; los hallazgos retroalimentan el diseño de los próximos componentes para minimizar breaking changes.

Nota del Arquitecto: El header `Sunset` en los responses HTTP es una de las prácticas más subestimadas en la gestión del ciclo de vida de APIs internas. Cuando los clientes instrumentados lo detectan —y es trivial hacerlo en cualquier middleware HTTP— el equipo consumidor recibe una señal automática de que está usando un componente en proceso de deprecación sin necesidad de que el equipo de plataforma contacte a cada equipo individualmente. El costo de añadir ese header es cero; el valor de tener visibilidad automática del uso de componentes deprecados es significativo.

El ciclo de vida de los componentes —desde su introducción hasta su deprecación— define la confiabilidad que los equipos consumidores pueden depositar en la plataforma. La siguiente sección examina cómo el equipo de plataforma incorpora el feedback de los equipos consumidores para asegurarse de que los componentes que introduce son los correctos y de que los que deprecas no se volverán a crear.

Módulo 10 – Capítulo 10 – Sección 04

Feedback loop con los equipos consumidores: cómo incorporar las necesidades de los usuarios de la plataforma

El riesgo más común en el desarrollo de plataformas internas es el mismo que en el desarrollo de productos externos: construir lo que parece valioso desde la perspectiva del equipo que lo construye, en lugar de lo que es realmente valioso para los equipos que lo usan. La diferencia en una plataforma interna es que no hay señal de mercado —los equipos consumidores son colegas, no clientes que votan con su dinero— y la invisibilidad del friction es especialmente alta: los ingenieros adaptan su forma de trabajar a las limitaciones de la plataforma, desarrollan workarounds que se vuelven parte del workflow normal, y dejan de reportar problemas porque asumen que son problemas inherentes que nadie puede resolver. Sin un sistema activo de captura de feedback, el equipo de plataforma opera en la oscuridad: no sabe qué workarounds están usando los equipos, no sabe qué abstracciones generan más fricción, y no sabe qué capacidades están bloqueando proyectos importantes.

El **feedback de alta frecuencia y bajo esfuerzo** es la primera capa del sistema: un canal de Slack dedicado (`#ai-platform-feedback` o `#platform-requests`) donde cualquier ingeniero puede reportar un pain point, solicitar una nueva capacidad, o preguntar una duda en cualquier momento, con un SLA de acknowledgment del equipo de plataforma de menos de 24 horas laborables. La respuesta no necesita ser una solución —puede ser "lo incluimos en el backlog del Q4" o "es un diseño intencional porque X, aquí está la alternativa Y"— pero debe existir. Sin acknowledgment, los canales de feedback se convierten en silos donde los reportes desaparecen sin respuesta, y los ingenieros dejan de reportar porque no sirve de nada. Las **adoption analytics** son el complemento no intrusivo del feedback explícito: instrumentación en el CLI y las APIs de la plataforma que registra qué comandos se usan más, qué endpoints reciben el mayor volumen, y qué operaciones generan más errores de usuario (mensajes de `invalid parameter`, `not found`, `permission denied` que indican friction en la experiencia de uso). Este feedback implícito —derivado del comportamiento real en lugar de las declaraciones de los usuarios— revela problemas que los usuarios no articulan porque no son conscientes de ellos.

El **feedback de mediana frecuencia y estructura media** es la encuesta trimestral de Developer Experience (DevEx): 5-10 preguntas con escala Likert sobre la satisfacción con cada componente principal de la plataforma (training cluster, model registry, serving, LLM gateway, feature store, documentación), una pregunta de NPS global ("¿con qué probabilidad recomendarías el uso de la plataforma de IA a un colega de otro equipo?"), y 1-2 preguntas abiertas sobre los pain points más importantes y las capacidades que más les faltan. Los resultados de la encuesta deben publicarse en el developer portal dentro de las dos semanas siguientes al cierre —incluyendo los resultados negativos— junto con el plan de respuesta del equipo de plataforma para los problemas más frecuentemente mencionados. Esta transparencia convierte la encuesta en un mecanismo de confianza, no solo de recolección de datos.

El **proceso de RFC (Request for Comments)** para nuevas capacidades importantes es el mecanismo que estructura el feedback de alta profundidad: cuando el equipo de plataforma planea introducir un nuevo componente o cambiar un componente existente de forma significativa, publica un documento RFC en el repositorio de la plataforma describiendo el problema que se quiere resolver, las alternativas consideradas, la solución propuesta, y las preguntas específicas sobre las que se quiere feedback. Los equipos consumidores tienen dos semanas para comentar en el PR del RFC. Este proceso tiene dos beneficios: primero, el feedback que llega antes del desarrollo es gratuito (cambiar el diseño de un documento es trivial; cambiar el diseño de un sistema implementado es costoso), y segundo, los equipos que participaron en el RFC son los primeros adoptantes naturales de la nueva capacidad, porque entienden el diseño y se sienten parte de él.

Las **office hours semanales** cierran el gap entre el feedback asíncrono del canal de Slack y la profundidad de las user research sessions: una sesión de 30-60 minutos, semanal o bisemanal, donde cualquier equipo puede reservar tiempo para traer preguntas técnicas, demos de problemas que están encontrando, o solicitudes de features con contexto de negocio. El formato en persona (o en videollamada) permite que el equipo de plataforma entienda el contexto completo del problema —que raramente se transmite en un mensaje de Slack— y que los equipos consumidores entiendan las restricciones técnicas o de prioridad que explican por qué ciertas solicitudes no están en el roadmap inmediato.

Mecanismos de feedback loop para plataformas de IA

- **Canal de Slack dedicado con SLA:** `#ai-platform-feedback` con SLA de acknowledgment de <24h laborables para cada reporte; cada item recibe respuesta visible

("en backlog Q4", "ya resuelto en v2.3", "diseño intencional porque X"); el canal funciona como historial público de la evolución de la plataforma.

- **Adoption analytics:** instrumentación en CLI y APIs que registra comandos, endpoints y operaciones con mayor uso, y operaciones que generan más errores de usuario; feedback implícito que revela friction que los usuarios no articulan explícitamente en el canal de feedback.
- **Encuesta trimestral de DevEx:** 5-10 preguntas Likert por componente + NPS global + preguntas abiertas; resultados publicados en el developer portal con plan de respuesta dentro de las dos semanas siguientes al cierre de la encuesta; incluir resultados negativos mantiene la credibilidad del proceso.
- **Proceso de RFC para nuevas capacidades:** documento de diseño publicado como PR en el repositorio de la plataforma con período de comentarios de dos semanas; captura feedback antes del desarrollo (gratis) y convierte a los participantes en early adopters con contexto suficiente.
- **Office hours semanales/biseminales:** sesión de 30-60 minutos donde equipos consumidores llevan preguntas, demos de problemas o solicitudes de features con contexto de negocio; permite profundidad de comprensión que no es posible en mensajes de texto asíncronos.
- **GitHub/GitLab issues públicos:** el repositorio de la plataforma acepta issues de feature requests y bug reports de todos los equipos consumidores con proceso visible de triage; la visibilidad del backlog genera confianza en que los problemas reportados tienen un destino.

Nota del Arquitecto: *El feedback loop tiene una característica crítica que distingue los procesos que funcionan de los que se convierten en teatro: el cierre visible. Los equipos consumidores necesitan poder atribuir directamente los cambios en la plataforma a los problemas que reportaron. Cuando el changelog de cada release incluye "resuelto: el error 'invalid model format' al registrar modelos de PyTorch (reportado por equipo de NLP en #platform-feedback)", el feedback se convierte en acción y la acción se hace visible. Esta atribución explícita es lo que mantiene la calidad y el volumen del feedback a lo largo del tiempo.*

El feedback loop transforma la relación entre el equipo de plataforma y los equipos consumidores de proveedor-cliente a colaboración bidireccional: los equipos consumidores son codesigners de la plataforma que usan. La siguiente sección examina cómo medir

objetivamente si esa colaboración está produciendo una plataforma que cumple su misión de multiplicar la productividad de los equipos de AI Engineering.

Módulo 10 – Capítulo 10 – Sección 05

Métricas de salud de la plataforma: adoption rate, time-to-deploy y platform incident rate

Una plataforma de IA sin métricas de salud opera en la oscuridad sobre su propio rendimiento: el equipo puede creer que está entregando valor porque los equipos consumidores no se quejan activamente, sin saber que en realidad los equipos han dejado de reportar problemas porque asumen que no habrá respuesta, o que están acumulando workarounds que hacen que la plataforma sea nominalmente usada pero efectivamente evitada para los casos de uso que más importan. Las métricas de salud de la plataforma son los indicadores objetivos que permiten distinguir entre una plataforma que está cumpliendo su misión de multiplicar la productividad de los equipos de AI Engineering y una que es formalmente existente pero que en la práctica no está siendo usada para las necesidades reales de los equipos.

El conjunto de métricas de salud se organiza en tres categorías que cubren dimensiones diferentes del valor de la plataforma. Las **métricas de adopción** miden si los equipos realmente usan la plataforma para sus necesidades reales, no solo en demos y proyectos piloto. El **adoption rate por componente** —porcentaje de equipos elegibles que han usado el componente al menos una vez en los últimos 30 días— distingue entre componentes maduros con adopción estabilizada y componentes nuevos en proceso de adopción. Un adoption rate estancado en el 40-50% para un componente maduro es una señal de que hay friction significativo que impide la adopción del resto de los equipos: puede ser documentación insuficiente, un modelo de permisos demasiado restrictivo, o una abstracción que no encaja con los casos de uso del 50% de los equipos que aún no adoptan. El **time-to-first-deploy para nuevos equipos** —tiempo en días calendario desde que un nuevo equipo es registrado en la plataforma hasta que completa su primer deployment exitoso de un modelo en producción— es el indicador más sensible de la calidad del onboarding: un target razonable es menos de 5 días laborables para un equipo que nunca ha usado la plataforma antes y que sigue la documentación de getting started sin ayuda adicional del equipo de plataforma.

Las **métricas de velocidad** miden si la plataforma realmente acelera el ciclo de desarrollo de los equipos de AI Engineering. El **deployment lead time** —tiempo en horas desde que se registra un nuevo modelo en el registry hasta que está sirviendo tráfico real en producción— incluye todos los pasos del proceso: la espera en la cola de revisión de governance, la ejecución de los automated gates de calidad, el proceso de promotion entre entornos, y el despliegue en el cluster de producción. Para deployments automatizados que no requieren revisión humana (modelos de bajo riesgo en la categorización del Capítulo 08), el target es menos de 4 horas. Para deployments que requieren revisión humana de governance (modelos de alto riesgo), el SLA del proceso de revisión debe estar explícitamente definido —por ejemplo, respuesta del revisor de governance en menos de 48 horas laborables. Estas métricas son la adaptación de las métricas DORA (DevOps Research and Assessment) al contexto de MLOps: deployment frequency de modelos (cuántos modelos entran a producción por semana por equipo), lead time for changes (el deployment lead time definido anteriormente), change failure rate (porcentaje de deployments de modelos que requieren rollback), y MTTR de incidentes de la plataforma.

Las **métricas de confiabilidad y satisfacción** miden la experiencia cualitativa de los equipos con la plataforma, que las métricas de velocidad y adopción no capturan completamente. El **platform-caused incident rate** —número de incidentes de producción donde la causa raíz es un fallo de la plataforma (no del modelo o la aplicación del equipo consumidor) por mes— es el indicador de confiabilidad más directo. Para que esta métrica sea fiable, cada incidente en el sistema de tracking debe ser clasificado con la causa raíz explícita y etiquetado con `caused-by: platform` cuando corresponde. El target para una plataforma madura es menos de un incidente causado por la plataforma por mes. El **Developer NPS** calculado trimestralmente de la encuesta de DevEx es el indicador de satisfacción que sintetiza la experiencia global de los equipos: la pregunta "¿con qué probabilidad (0-10) recomendarías el uso de la plataforma de IA de la organización a un colega de otro equipo?" produce un NPS que puede compararse entre trimestres y que señala tendencias de mejora o deterioro en la percepción de la plataforma. El target para una plataforma madura es un NPS superior a +40.

La publicación mensual de todas estas métricas en el developer portal —con comparación respecto al trimestre anterior y objetivos explícitos para el próximo trimestre— es la práctica que convierte las métricas en instrumento de confianza: los equipos consumidores pueden ver que el equipo de plataforma se mide por los mismos criterios que ellos usan para evaluar si la plataforma vale la inversión de adopción.

Métricas clave de salud de la plataforma

- **Adoption rate por componente:** porcentaje de equipos elegibles con uso activo en los últimos 30 días; calculado automáticamente por los logs de uso de la plataforma; target > 80% para componentes maduros (>6 meses disponibles); adoption rate estancado indica friction de adopción que requiere investigación.
- **Time-to-first-deploy (onboarding metric):** tiempo en días calendario desde el registro del nuevo equipo hasta el primer deployment exitoso en producción siguiendo únicamente la documentación pública; target < 5 días laborables; deterioro en esta métrica señala documentación desactualizada o friction en el proceso de onboarding.
- **Deployment lead time (adaptación DORA):** tiempo en horas desde el registro del modelo en el registry hasta que sirve tráfico en producción; target < 4 horas para deployments automatizados (bajo riesgo), SLA explícito para deployments con revisión humana de governance.
- **Platform-caused incident rate:** número de incidentes de producción con `caused-by: platform` en el sistema de tracking por mes; target < 1/mes para plataformas maduras; requiere clasificación explícita de causa raíz en cada incidente del historial.
- **Developer NPS (DevEx Survey):** Net Promoter Score trimestral de la encuesta de Developer Experience; target > +40 para plataformas maduras; la tendencia trimestral es más informativa que el valor absoluto en un trimestre aislado.

Nota del Arquitecto: La transparencia de las métricas de salud de la plataforma es en sí misma un mecanismo de confianza: cuando los equipos consumidores pueden ver que el adoption rate del feature store bajó del 82% al 71% y que el equipo de plataforma tiene un plan de respuesta publicado, la plataforma comunica que se toma en serio su misión. La opacidad —publicar solo las métricas que son buenas— destruye la credibilidad del proceso de medición completo. Las métricas que nadie quiere ver son precisamente las que más importan rastrear y publicar.

Las métricas de salud de la plataforma cierran el ciclo de feedback cuantitativo que complementa el feedback cualitativo de las encuestas de DevEx y las office hours: juntos, proveen la visión completa que el equipo de plataforma necesita para priorizar dónde invertir su capacity. La sección de cierre de este capítulo —y del módulo— sintetiza las tensiones fundamentales que definen la operación continua de una plataforma de IA y el principio rector que las gobierna.

Módulo 10 – Capítulo 10 – Sección 06

Cierre: una plataforma de IA no se construye una vez, se opera y evoluciona continuamente

Una plataforma de IA alcanza su punto de mayor valor no en el día de su lanzamiento, sino después de años de iteración impulsada por el feedback real de los equipos que la usan, por la reducción sistemática de la deuda técnica acumulada, y por la incorporación de nuevas capacidades que responden a los cambios en el ecosistema de IA. Esta verdad tiene una consecuencia incómoda para la planificación: no hay un punto de llegada donde la plataforma esté "terminada" y el equipo pueda pasar a otra cosa. Los modelos base que eran state-of-the-art hace doce meses son hoy los modelos económicos de segunda categoría; los frameworks de serving que eran la elección estándar están siendo reemplazados por alternativas más eficientes; las herramientas de observabilidad específicas para LLMs no existían hace tres años. Una plataforma de IA que no evoluciona con el ecosistema se convierte, de forma gradual pero inevitable, en una restricción en lugar de un habilitador: los equipos de AI Engineering empiezan a bypassarla para usar directamente las nuevas herramientas porque la plataforma no las soporta todavía, y en el momento en que los equipos empiezan a trabajar alrededor de la plataforma, la plataforma ha perdido su razón de ser.

La **sostenibilidad operacional** de una plataforma de IA requiere gestionar en equilibrio tres tensiones que son permanentes y que no pueden resolverse definitivamente, solo gestionarse continuamente. La primera es la tensión entre **estabilidad y evolución**: los equipos consumidores necesitan que las abstracciones que construyeron encima de la plataforma sigan funcionando de forma predecible (estabilidad), pero la plataforma necesita incorporar nuevas capacidades y actualizar sus componentes para seguir siendo relevante (evolución). Esta tensión se gestiona con políticas explícitas de backward compatibility, versionado semántico, y procesos de deprecación con tiempo suficiente —exactamente los mecanismos que las secciones anteriores de este capítulo han descrito. La segunda es la tensión entre **velocidad de entrega y calidad técnica**: la presión para entregar nuevas capacidades que los equipos solicitan puede llevar a acumular deuda técnica que hace que la velocidad de entrega se deteriore progresivamente hasta que el equipo de plataforma pasa

más tiempo en mantenimiento que en nuevas capacidades. Esta tensión se gestiona con la reserva explícita del 20-30% del capacity del equipo para reducción de deuda, protegida del debate con los stakeholders con el mismo rigor con que se protegen los compromisos de entrega. La tercera es la tensión entre **centralización y autonomía**: una plataforma que centraliza demasiado hace que los equipos consumidores dependan de ella para cada pequeña decisión técnica, creando un cuello de botella que degrada la autonomía y la velocidad de los equipos; una plataforma que centraliza demasiado poco no provee los beneficios de estandarización y economía de escala que justifican su existencia. Esta tensión se gestiona con el modelo de "golden paths" que el Capítulo 02 introdujo: el camino por defecto está optimizado y es recomendado, pero no es obligatorio para casos de uso que genuinamente lo justifican.

El **modelo organizacional** que sostiene estas tres tensiones es el Team Topologies de Enabling Team y Platform Team que el Capítulo 01 introdujo: el equipo de plataforma no es el guardián que aprueba o rechaza lo que los equipos hacen —es el equipo que reduce la carga cognitiva de los equipos de AI Engineering para que puedan enfocarse en el valor de negocio de sus aplicaciones de IA. Esta identidad de equipo habilitador, no equipo de control, es la que determina si la plataforma se convierte en un acelerador genuino o en una capa burocrática adicional.

El módulo completo ha construido el mapa de una plataforma de IA funcionando en producción: desde la arquitectura conceptual y los principios de diseño del Capítulo 01, a través de la arquitectura técnica de cinco planos del Capítulo 02, el model registry del Capítulo 03, las pipelines de datos del Capítulo 04, el monitoring y observabilidad del Capítulo 05, las pipelines de entrenamiento continuo del Capítulo 06, el LLM Gateway del Capítulo 07, el gobierno y compliance del Capítulo 08, la gestión de costos del Capítulo 09, y finalmente la operación continua, la gestión de la deuda técnica, y la evolución sostenible del Capítulo 10. El Módulo 11 — Enterprise AI — extiende esta arquitectura hacia la escala de las organizaciones más grandes: múltiples plataformas regionales coordinadas, estrategias de AI Center of Excellence, y los patrones de gobernanza que permiten que decenas de equipos de AI Engineering trabajen sobre la misma infraestructura con autonomía suficiente y coordinación central cuando importa.

Principio rector

Una plataforma de IA es un ser vivo que requiere cuidado continuo. El equipo que la construye debe tratar cada semana como si fuera la primera semana del producto: con la

misma atención a las necesidades reales de los usuarios, con la misma disposición a cambiar lo que no funciona, y con la misma humildad de saber que el diseño de hoy es el punto de partida de la refactorización de mañana.

"Software is never finished, only abandoned."

— Atribuida a la cultura de ingeniería de software. La afirmación captura la verdad operacional más importante de este módulo: los sistemas de software que generan valor duradero no se "terminan" —se operan, se evolucionan, y se cuidan con la misma atención que los productos de negocio que los consumen. Una plataforma de IA es exactamente eso: un producto técnico vivo cuyo ciclo de vida se mide en años, no en sprints.